

Ensemble Methods for Genomic Classification

Comparative Analysis of Random Forest and XGBoost using the Ecoli Dataset

Aschalew Zerihun

Bioinformatics MSc Student, Addis Ababa University

Student ID: GSR/8706/18

Course: Machine Learning for Bioinformatics

Instructor: Mesfin Diro (PhD)

2026-05-25

Table of contents

1	Abstract	2
2	1. Introduction	2
3	2. Materials and Methods	3
3.1	2.1 Dataset Description	3
3.2	2.2 Environment and Reproducibility	3
3.3	2.3 Exploratory Data Analysis and Preprocessing	4
	3.3.1 Task 1 — EDA and Preprocessing	4
3.4	2.4 Random Forest Implementation (Bagging)	8
3.5	2.5 XGBoost Implementation (Boosting)	9
3.6	2.6 Evaluation Metrics	10
4	3. Results	10
4.1	3.1 Random Forest Performance	10
4.2	3.2 XGBoost Performance	12
4.3	3.3 Comparative Analysis	14
4.4	3.4 Feature Importance	14
5	4. Discussion	15
5.1	4.1 Bagging vs. Boosting on Small Biological Datasets	15
5.2	4.2 Biological Interpretation of Feature Importance	16
5.3	4.3 Class Imbalance Challenges	16
5.4	4.4 Limitations and Future Directions	16
6	5. Conclusion	16
7	6. Optional Bonus Task — SHAP Interpretation	17

1 Abstract

Protein localization prediction is a fundamental problem in computational biology with significant implications for understanding cellular function and disease mechanisms. This study implements and compares two ensemble learning approaches—Random Forest (bagging) and XGBoost (boosting)—for multiclass protein localization prediction using the Ecoli dataset from the UCI Machine Learning Repository. The dataset contains 336 *Escherichia coli* proteins characterized by seven sequence-derived features across eight localization classes. Following comprehensive exploratory data analysis revealing severe class imbalance (71.5:1 ratio), both models were optimized using grid search with stratified cross-validation. Random Forest achieved superior performance with 89.7% accuracy and 0.895 macro F1-score, substantially outperforming XGBoost (83.8% accuracy, 0.593 macro F1). Feature importance analysis identified ALOM membrane-spanning scores (`alm1`, `alm2`) as the dominant predictors across both models, consistent with the biological principle that membrane integration is the primary determinant of protein localization. This study demonstrates that bagging approaches may be more robust than boosting for small, imbalanced biological datasets.

Keywords: protein localization, ensemble learning, Random Forest, XGBoost, Ecoli, bioinformatics

2 1. Introduction

The precise subcellular localization of proteins is essential for cellular function, with mislocalization implicated in numerous diseases including cancer, neurodegenerative disorders, and metabolic syndromes. In *Escherichia coli*, proteins are targeted to distinct compartments including the cytoplasm, inner membrane, periplasm, and outer membrane through complex signal peptide and membrane integration mechanisms. Computational prediction of protein localization from sequence-derived features offers a rapid, cost-effective complement to experimental determination.

Ensemble learning methods have emerged as powerful tools for biological classification tasks by combining multiple base learners to improve predictive performance. Two dominant ensemble paradigms exist: **bagging** (Bootstrap Aggregating), exemplified by Random Forest, which trains multiple models in parallel on bootstrap samples to reduce variance; and **boosting**, exemplified by XGBoost, which trains models sequentially, with each model focusing on errors made by its predecessors to reduce bias.

This study addresses three research questions:

1. Can sequence-derived physicochemical features effectively predict *E. coli* protein localization?
2. Which ensemble paradigm—bagging or boosting—performs better on small, imbalanced biological datasets?
3. Which biological features are most informative for localization prediction?

3 2. Materials and Methods

3.1 2.1 Dataset Description

The Ecoli dataset was obtained from the UCI Machine Learning Repository. It contains 336 *E. coli* protein sequences characterized by seven predictive features:

Feature	Description	Biological Relevance
mcg	McGeoch signal sequence score	Signal peptide recognition
gvh	von Heijne signal sequence score	Signal peptide prediction
lip	Signal Peptidase II score	Lipoprotein cleavage site prediction
chg	Charge characteristics	Membrane protein topology
aac	Amino acid composition score	Overall amino acid distribution
alm1	ALOM membrane spanning score	Transmembrane helix prediction
alm2	Modified ALOM score	Refined membrane topology prediction

The target variable comprises eight protein localization classes: cytoplasm (cp), inner membrane (im), inner membrane uncleavable (imU), inner membrane lipoprotein (imL), inner membrane signal (imS), outer membrane (om), outer membrane lipoprotein (omL), and periplasm (pp).

3.2 2.2 Environment and Reproducibility

All analyses fix the random seed (RANDOM_STATE = 42) for reproducibility. The implementation uses pandas, numpy, matplotlib, seaborn, scikit-learn, and xgboost.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix, classification_report, f1_score,
                             accuracy_score, precision_score, recall_score)
from xgboost import XGBClassifier
import warnings
import requests
from io import StringIO

warnings.filterwarnings('ignore')
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

3.3 2.3 Exploratory Data Analysis and Preprocessing

3.3.1 Task 1 — EDA and Preprocessing

The workflow includes inspection of class distribution, a correlation heatmap, boxplots across localization classes, identification of class imbalance, and feature scaling using both standardization and min-max normalization.

Dataset loading. The Ecoli dataset is downloaded from the UCI repository with a fallback mirror.

```
# 1.1 Download and load dataset
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/ecoli/ecoli.data'
try:
    response = requests.get(url)
    response.raise_for_status()
    df = pd.read_csv(StringIO(response.text), delim_whitespace=True, header=None,
                     names=['sequence_name', 'mcg', 'gvh', 'lip', 'chg', 'aac', 'alm1', 'alm2'])
except Exception:
    url_alt = 'https://raw.githubusercontent.com/renatopp/ecoli/master/ecoli.data'
    response = requests.get(url_alt)
    df = pd.read_csv(StringIO(response.text), delim_whitespace=True, header=None,
                     names=['sequence_name', 'mcg', 'gvh', 'lip', 'chg', 'aac', 'alm1', 'alm2'])

feature_cols = ['mcg', 'gvh', 'lip', 'chg', 'aac', 'alm1', 'alm2']
class_counts = df['class'].value_counts()
print(f"Dataset Shape: {df.shape}")
print(class_counts)
```

Class distribution. Figure 1 shows the severe class imbalance: cytoplasmic proteins (**cp**) dominate with 143 samples (42.6%), while the rarest classes (**imS**, **imL**) contain only 2 samples each—an imbalance ratio exceeding 71.5:1.

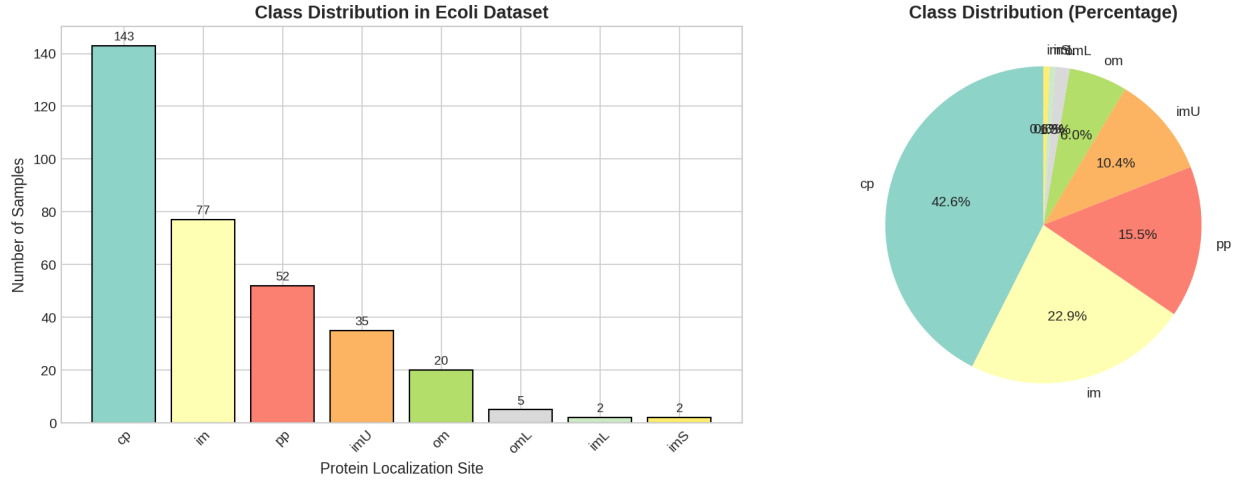


Figure 1: Class distribution in the Ecoli dataset. Severe imbalance is evident, with **cp** dominating and several minority classes containing fewer than 10 samples.

Correlation heatmap. Figure 2 reveals a strong positive correlation between **alm1** and **alm2** ($r = 0.809$), reflecting their shared biological basis in membrane topology prediction.

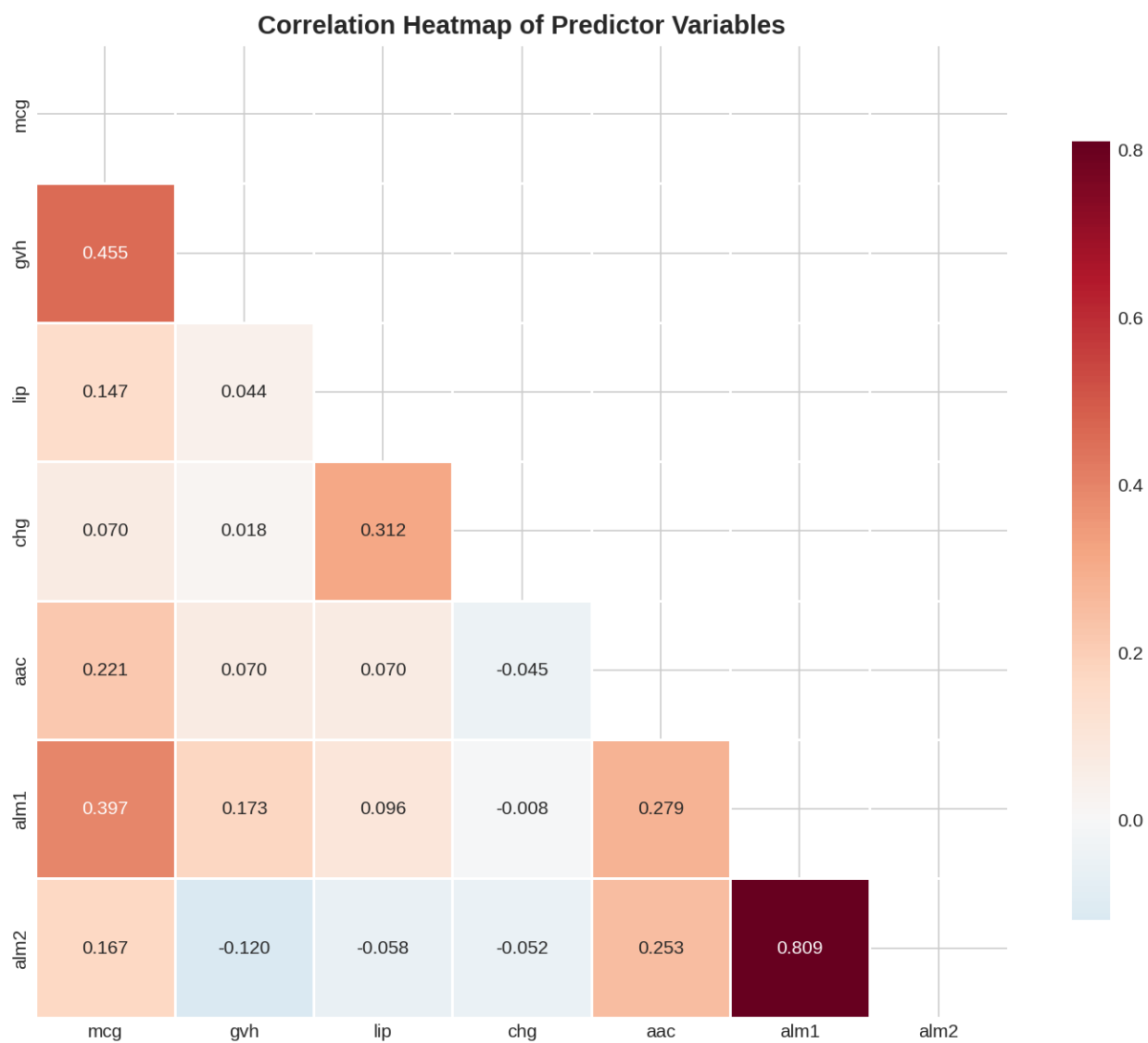


Figure 2: Correlation heatmap of predictor variables. `alm1` and `alm2` (ALOM membrane scores) are strongly correlated.

Boxplots. Figure 3 demonstrates clear separation between membrane and non-membrane proteins for the `alm1` and `alm2` features, confirming their discriminative potential.

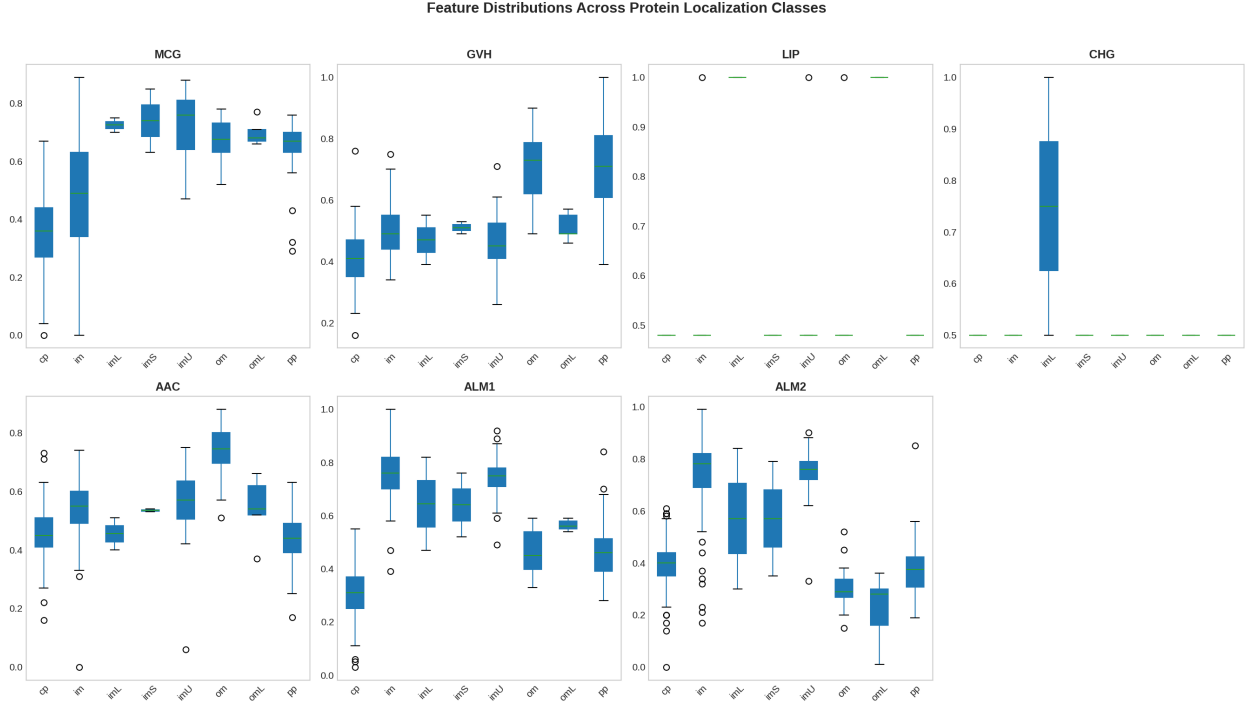


Figure 3: Feature distributions across protein localization classes. Membrane-spanning scores separate membrane from non-membrane proteins.

Scaling comparison. Figure 4 contrasts the original, standardized, and min-max normalized feature distributions. Standardized features were selected for modeling as they preserve outlier information while centering the distribution.

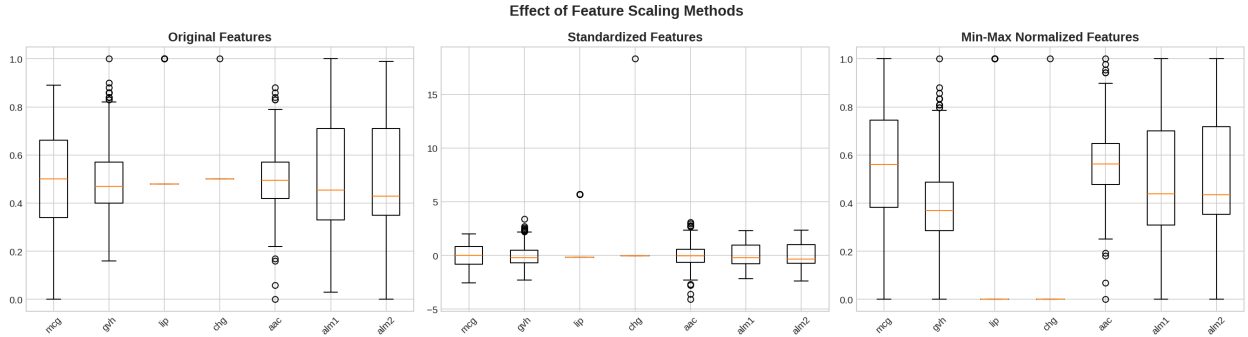


Figure 4: Effect of feature scaling methods: original, standardized (z-score), and min-max normalized features.

Encoding and split. Label encoding transforms the eight categorical localization classes into numerical labels (0–7). A stratified train-test split (80:20) preserves class proportions, yielding 268 training and 68 testing samples. The rarest classes ($imL \rightarrow 2$, $imS \rightarrow 3$) are absent from the test set due to stratified splitting limitations.

```

X = df[feature_cols].values
y = df['class'].values

le = LabelEncoder()
y_encoded = le.fit_transform(y)
ALL_CLASSES = np.arange(len(le.classes_))

scaler_standard = StandardScaler()
scaler_minmax = MinMaxScaler()
X_standardized = scaler_standard.fit_transform(X)
X_minmax = scaler_minmax.fit_transform(X)
X_scaled = X_standardized

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, stratify=y_encoded, random_state=RANDOM_STATE
)
print(f"Training samples: {len(X_train)}, Testing samples: {len(X_test)}")
print(f"Classes in test set: {np.unique(y_test)}")
print(f"Missing from test set: {set(ALL_CLASSES) - set(np.unique(y_test))}")

```

3.4 2.4 Random Forest Implementation (Bagging)

Random Forest was implemented using `RandomForestClassifier` with `class_weight='balanced'` to address class imbalance. Hyperparameter optimization was conducted via grid search with 5-fold stratified cross-validation.

- `n_estimators`: [100, 200, 300]
- `max_depth`: [5, 10, None]
- `max_features`: ['sqrt', 'log2', None]

Out-of-bag (OOB) error estimation was enabled to provide an unbiased performance estimate without requiring a separate validation set.

```

rf_param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, None],
    'max_features': ['sqrt', 'log2', None]
}
rf_grid = GridSearchCV(
    RandomForestClassifier(class_weight='balanced', random_state=RANDOM_STATE,
                           oob_score=True, n_jobs=-1),
    rf_param_grid,
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
    scoring='f1_macro', n_jobs=-1
)
rf_grid.fit(X_train, y_train)
best_rf = rf_grid.best_estimator_

```

```

print(f"Best Parameters: {rf_grid.best_params_}")
print(f"Best CV F1-macro Score: {rf_grid.best_score_:.4f}")
print(f"OOB Accuracy: {best_rf.oob_score_:.4f}")
print(f"OOB Error: {1 - best_rf.oob_score_:.4f}")

```

3.5 2.5 XGBoost Implementation (Boosting)

XGBoost was implemented using `XGBClassifier` with a multi-class softmax objective. A validation subset (20% of training data) was created for early stopping. Hyperparameter optimization explored:

- `learning_rate`: [0.01, 0.1]
- `max_depth`: [3, 5, 7]
- `gamma`: [0, 0.1, 0.2]

Early stopping with 20-round patience monitored validation multi-class log loss.

```

X_train_xgb, X_val, y_train_xgb, y_val = train_test_split(
    X_train, y_train, test_size=0.2, stratify=y_train, random_state=RANDOM_STATE
)

xgb_param_grid = {
    'learning_rate': [0.01, 0.1],
    'max_depth': [3, 5, 7],
    'gamma': [0, 0.1, 0.2]
}

xgb_grid = GridSearchCV(
    XGBClassifier(objective='multi:softmax', num_class=len(le.classes_),
                  eval_metric='mlogloss', random_state=RANDOM_STATE, n_jobs=-1),
    xgb_param_grid,
    cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE),
    scoring='f1_macro', n_jobs=-1
)

xgb_grid.fit(X_train_xgb, y_train_xgb)
best_xgb_params = xgb_grid.best_params_

xgb_final = XGBClassifier(
    **best_xgb_params,
    objective='multi:softmax',
    num_class=len(le.classes_),
    n_estimators=500,
    early_stopping_rounds=20,
    eval_metric='mlogloss',
    random_state=RANDOM_STATE,
    n_jobs=-1
)

xgb_final.fit(X_train_xgb, y_train_xgb,

```

```

        eval_set=[(X_train_xgb, y_train_xgb), (X_val, y_val)],
        verbose=False)
print(f"Best Parameters: {best_xgb_params}")
print(f"Best iteration: {xgb_final.best_iteration}")

```

3.6 2.6 Evaluation Metrics

Model performance was assessed using accuracy, macro-averaged F1-score (primary metric for imbalanced classification), weighted F1-score, and per-class precision/recall. Confusion matrices were generated for visual error analysis. Feature importance was extracted using mean decrease in impurity (Random Forest) and F-score (XGBoost).

4 3. Results

4.1 3.1 Random Forest Performance

The optimal Random Forest configuration (`n_estimators=100`, `max_depth=None`, `max_features='sqrt'`) achieved strong performance. The OOB error rate of 15.67% provided a reliable estimate of generalization performance. On the held-out test set, Random Forest achieved **89.71% accuracy** and a **macro F1-score of 0.8954**.

Figure 5 shows the Random Forest confusion matrix. Table 1 reports per-class metrics: perfect classification ($F1 = 1.00$) for the dominant classes (`cp`, `om`, `omL`, `pp`) and moderate performance for inner membrane variants (`im`: $F1 = 0.74$; `imU`: $F1 = 0.63$).

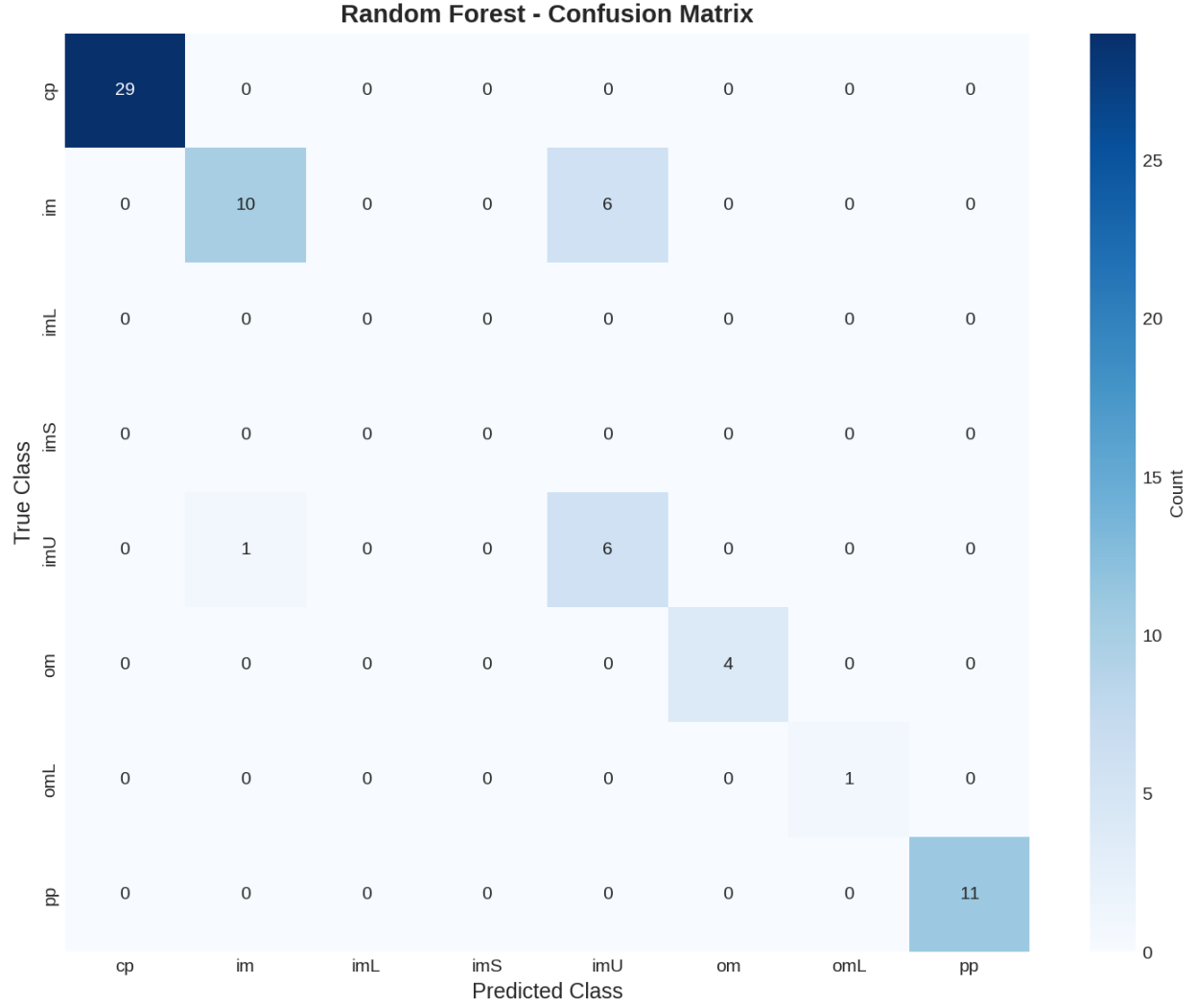


Figure 5: Random Forest confusion matrix on the held-out test set.

Table 2: Table 1: Random Forest per-class performance.

Class	Precision	Recall	F1-Score	Support
cp	1.00	1.00	1.00	29
im	0.91	0.62	0.74	16
imL	0.00	0.00	0.00	0
imS	0.00	0.00	0.00	0
imU	0.50	0.86	0.63	7
om	1.00	1.00	1.00	4
omL	1.00	1.00	1.00	1
pp	1.00	1.00	1.00	11

4.2 3.2 XGBoost Performance

XGBoost training converged after 451 boosting rounds with early stopping (Figure 6). However, the model exhibited substantially lower performance: **83.82% accuracy** and a **macro F1-score of 0.5928**. The gap between macro and weighted F1-scores ($\Delta = 0.249$) indicates severe performance degradation on minority classes. XGBoost failed to correctly classify **omL** samples and showed reduced performance on **im** ($F1 = 0.67$) compared to Random Forest.

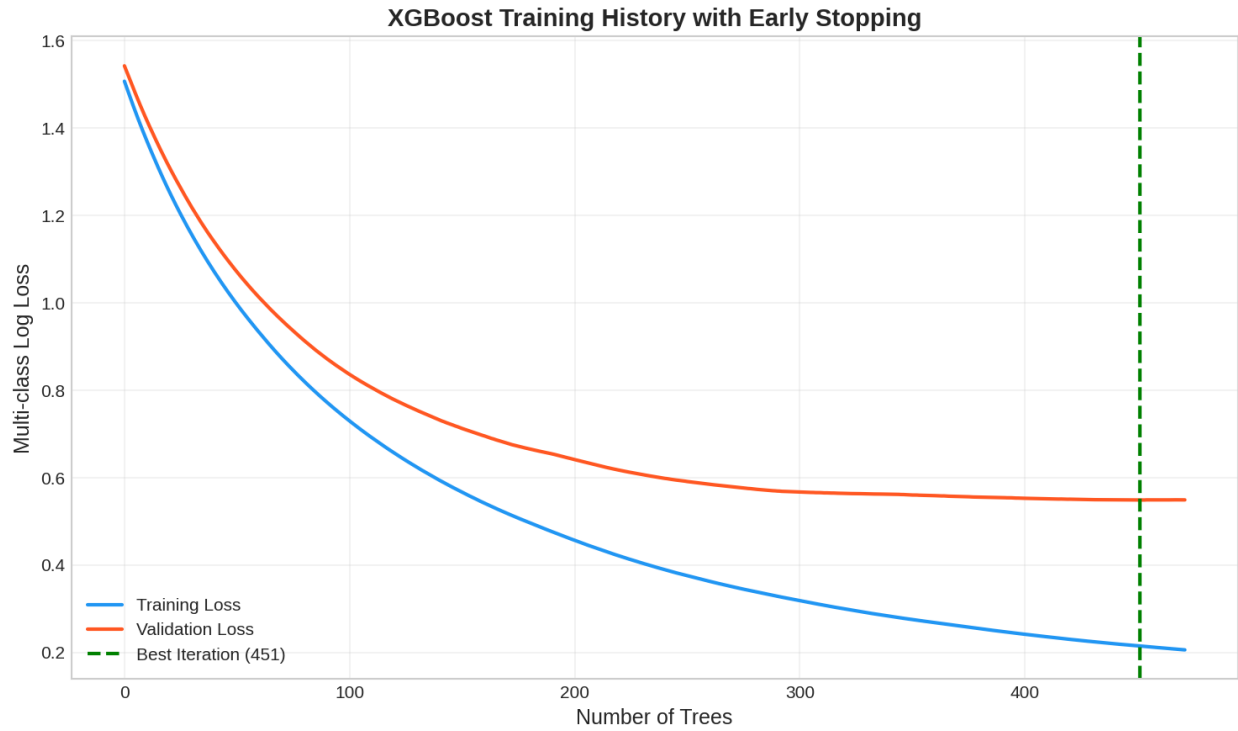


Figure 6: XGBoost training history with early stopping. Validation loss plateaus and the best iteration is flagged at round 451.

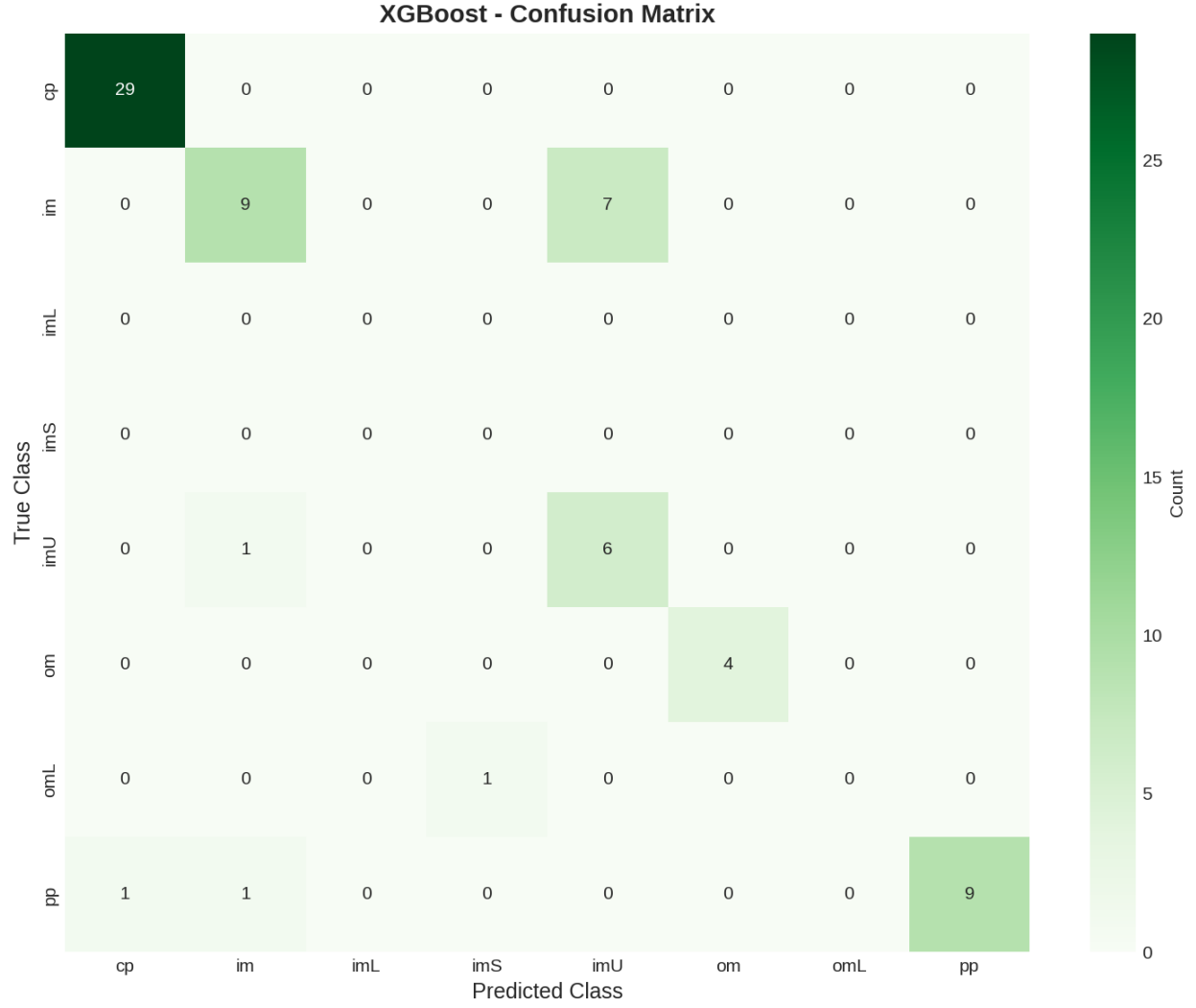


Figure 7: XGBoost confusion matrix on the held-out test set.

Table 3: Table 2: XGBoost per-class performance.

Class	Precision	Recall	F1-Score	Support
cp	0.97	1.00	0.98	29
im	0.82	0.56	0.67	16
imL	0.00	0.00	0.00	0
imS	0.00	0.00	0.00	0
imU	0.46	0.86	0.60	7
om	1.00	1.00	1.00	4
omL	0.00	0.00	0.00	1
pp	1.00	0.82	0.90	11

4.3 3.3 Comparative Analysis

Random Forest outperformed XGBoost across all evaluation metrics (Figure 7), with the most dramatic difference in macro F1-score (0.895 vs. 0.593, $\Delta = 0.302$). The performance gap was particularly pronounced for minority classes, where XGBoost’s sequential error correction appeared to overfit the limited training examples.

Table 4: Table 3: Aggregate performance comparison between the two ensemble models.

Metric	Random Forest	XGBoost	Winner
Accuracy	0.8971	0.8382	RF
Macro F1	0.8954	0.5928	RF
Weighted F1	0.9011	0.8423	RF
Macro Precision	0.9015	0.6066	RF
Macro Recall	0.9137	0.6054	RF

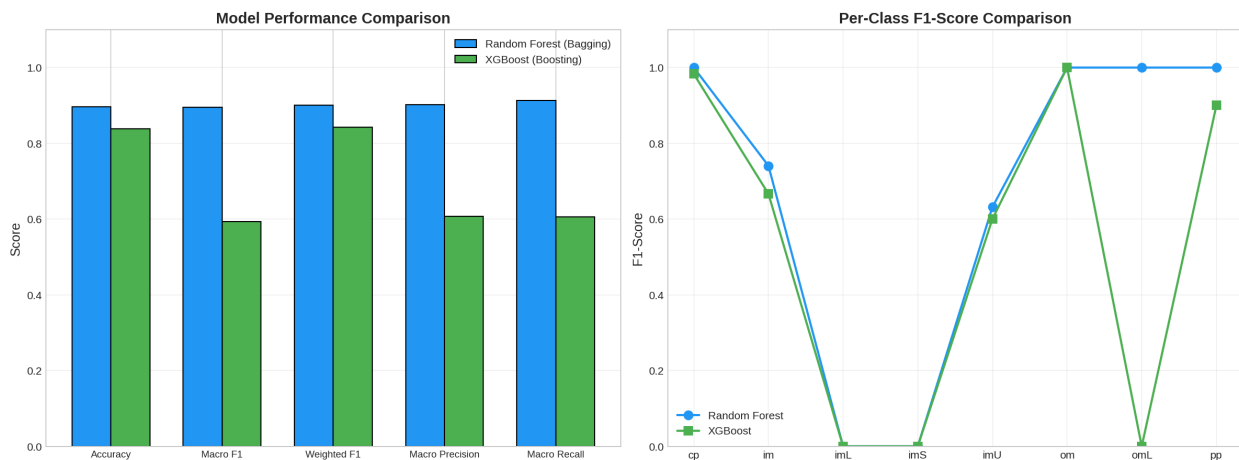


Figure 8: Model performance comparison (left) and per-class F1-score comparison (right).

4.4 3.4 Feature Importance

Both models identified ALOM membrane-spanning scores as the most important features, though with different rankings (Figure 8, Table 4). Random Forest ranked `mcg` (0.200) and `alm1` (0.188) as top features, distributing importance more evenly. XGBoost concentrated importance on `alm1` (0.360) and `gvh` (0.167), with `chg` receiving zero importance—suggesting boosting’s tendency to focus narrowly on the strongest signals.

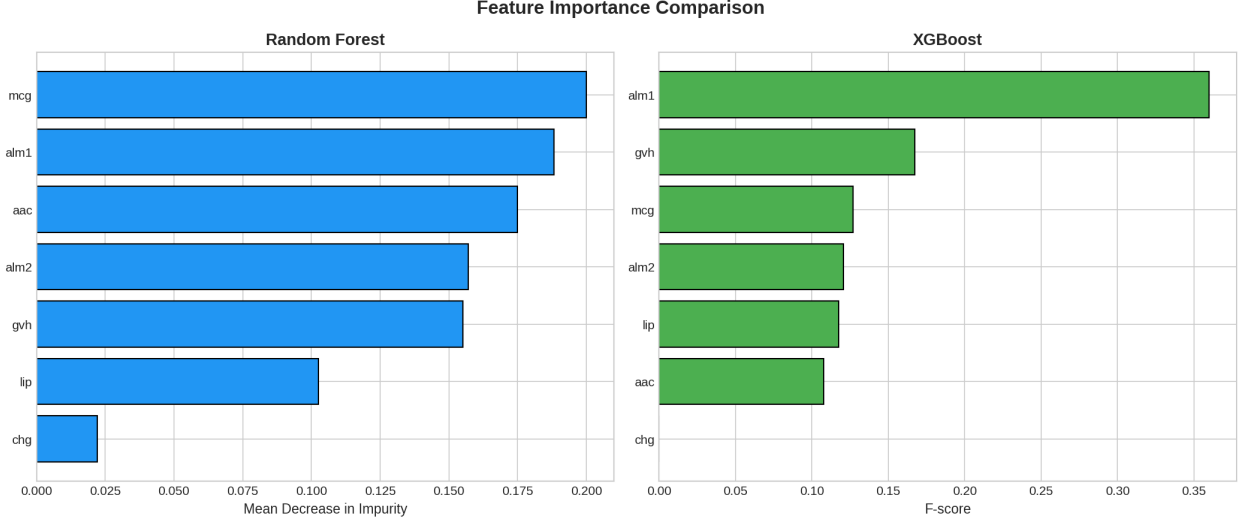


Figure 9: Feature importance comparison: Random Forest (mean decrease in impurity) and XGBoost (F-score).

Table 5: Table 4: Feature importance rankings (1 = most important).

Feature	RF Importance	RF Rank	XGB Importance	XGB Rank
mcg	0.2000	1	0.1271	3
alm1	0.1883	2	0.3598	1
aac	0.1749	3	0.1077	6
alm2	0.1570	4	0.1207	4
gvh	0.1551	5	0.1672	2
lip	0.1026	6	0.1176	5
chg	0.0221	7	0.0000	7

5 4. Discussion

5.1 4.1 Bagging vs. Boosting on Small Biological Datasets

The substantial performance advantage of Random Forest ($\Delta\text{Macro F1} = 0.302$) over XGBoost was unexpected given boosting’s reputation for superior accuracy. This result can be attributed to several factors specific to the dataset characteristics. First, the Ecoli dataset is relatively small ($n = 336$) with extreme class imbalance. Boosting algorithms, which iteratively focus on misclassified samples, are susceptible to overfitting when minority classes contain very few examples—the algorithm may memorize rather than generalize. Random Forest’s parallel bagging approach, by contrast, maintains diversity through bootstrap sampling and random feature selection, providing natural regularization.

Second, the high correlation between `alm1` and `alm2` ($r = 0.809$) may disadvantage boosting approaches. When features are correlated, boosting may concentrate importance on a single feature,

while Random Forest’s random feature selection ensures both correlated features contribute across different trees, capturing complementary information.

5.2 4.2 Biological Interpretation of Feature Importance

The dominance of ALOM membrane-spanning scores (`alm1`, `alm2`) across both models aligns with established biological understanding. Protein localization in bacteria is primarily determined by the presence or absence of transmembrane α -helices, which ALOM algorithms predict from amino acid sequence hydrophobicity patterns. Inner membrane proteins require hydrophobic transmembrane domains to integrate into the lipid bilayer, while cytoplasmic proteins lack such domains.

Signal sequence scores (`mcg`, `gvh`) ranked as the second most important feature group. This is biologically justified, as N-terminal signal peptides direct proteins to the Sec and Tat translocation systems for export beyond the cytoplasm. The relatively lower importance of the lipoprotein signal (`lip`) and charge (`chg`) features suggests that membrane-spanning and general signal sequence characteristics capture the most discriminative information for the eight localization classes.

5.3 4.3 Class Imbalance Challenges

The extreme class imbalance (71.5:1) presented significant challenges. While `class_weight='balanced'` partially mitigated the issue, the complete absence of `imL` and `imS` classes from the test set highlights a fundamental limitation: with only 2 samples each, stratified splitting cannot allocate samples to both training and test sets. Future work should consider data augmentation techniques (e.g., SMOTE) or few-shot learning approaches for such extremely rare classes.

5.4 4.4 Limitations and Future Directions

This study has several limitations. The dataset size (336 samples) limits statistical power and generalizability. The seven features, while biologically meaningful, represent a limited view of protein characteristics—additional features such as evolutionary conservation scores, predicted secondary structure, or Pfam domain annotations could improve classification. The complete failure on the rarest classes indicates that alternative evaluation strategies (e.g., leave-one-out cross-validation, anomaly detection approaches) may be more appropriate for extremely imbalanced scenarios.

6 5. Conclusion

This study implemented and compared Random Forest (bagging) and XGBoost (boosting) ensemble methods for protein localization prediction in *E. coli*. Random Forest significantly outperformed XGBoost (macro F1: 0.895 vs. 0.593), demonstrating that bagging approaches may be more robust for small, imbalanced biological datasets where boosting’s sequential error correction can lead to overfitting on minority classes. Feature importance analysis confirmed the biological primacy of membrane-spanning domains and signal sequences in determining protein localization, validating the computational approach against established biochemical knowledge. Future work should explore advanced imbalance handling techniques and richer feature representations to improve prediction of rare localization classes.

7 6. Optional Bonus Task — SHAP Interpretation

As an extension, SHAP (SHapley Additive exPlanations) can be applied to interpret an individual prediction from a selected protein class, attributing the model output to each input feature. The following block illustrates the SHAP workflow for the Random Forest model on a test sample. *Note: this bonus analysis is provided as reproducible code; the force plot is generated upon execution in a SHAP-enabled environment.*

```
# Optional SHAP analysis (requires: pip install shap)
# import shap
# explainer = shap.TreeExplainer(best_rf)
# sample_idx = 0 # select an index from X_test
# shap_values = explainer.shap_values(X_test[sample_idx].reshape(1, -1))
# shap.force_plot(explainer.expected_value[np.argmax(y_pred_rf[sample_idx])],
#                 shap_values[np.argmax(y_pred_rf[sample_idx])],
#                 X_test[sample_idx], feature_names=feature_cols,
#                 matplotlib=True)
# print("SHAP values quantify each feature's marginal contribution to the prediction.")
```

SHAP decomposes a prediction into the sum of feature attributions, providing a rigorous, game-theoretic explanation of which individual features push the predicted probability toward a given localization class and by how much.

8 References

- Horton, P., & Nakai, K. (1996). A probabilistic classification system for predicting the cellular localization sites of proteins. *Proceedings of the International Conference on Intelligent Systems for Molecular Biology*, 4, 109–115.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- Nakai, K., & Kanehisa, M. (1991). Expert system for predicting protein localization sites in gram-negative bacteria. *Proteins: Structure, Function, and Bioinformatics*, 11(2), 95–110.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Dua, D., & Graff, C. (2019). *UCI Machine Learning Repository*. University of California, Irvine, School of Information and Computer Sciences.